

Technical & User Documentation

DAC Executive Dashboard - how to use it, run it, and maintain it

Document	Technical / User Documentation
Application	Con Edison Disadvantaged Communities (DAC) Executive Dashboard
Audience	Part 1 (Users): report owners and editors. Part 2 (Technical): developers / maintainers.
Environment	Clara Fortuna Dev (Developer environment with Dataverse) - TEST build
Status	Current as of June 24, 2026. Production move is a separate, future phase.

1. What this application is

The DAC Executive Dashboard is a web application that presents Con Edison's annual Disadvantaged Communities (DAC) report - ten regulatory sections (A through J) of tables, KPIs, charts, and a service-territory map. Authorized users can edit the underlying numbers directly in the app, and every change is stored and audited in Microsoft Dataverse.

It is hosted inside Microsoft Power Apps as a model-driven app and is served from Dataverse web resources. The current build runs in a development/test environment; the mechanism is identical to what production will use.

Part 1 - User guide

2. Signing in

1. Open the dashboard. In the test environment the entry URL is the password gate:
[.../WebResources/cr2bf_dactest/index.html](#).
2. Enter the access password: [dacdemo](#), to proceed to the dashboard.

Tip: If the dashboard does not load correctly after opening it directly, start from the gate page (index.html) to run the standard entry flow.

3. Finding your way around

The dashboard is organized by the report's ten sections. For each section you can see its tables, the KPIs and totals, and the relevant charts. A service-territory map shows DAC and non-DAC areas across Con Edison's territory.

Area	What you do there
Sections (A-J)	Browse each report area - Clean Energy Spending, EV Make-Ready, Demand Response, DER, Capital Investments, Outages, Main Replacement, Leak Repairs, Clean Energy Jobs, Customer Operations.
KPIs & totals	See headline figures and sums; totals are computed from the current data.
Charts	Visual summaries per section; hover for tooltips with the underlying values.
Map	Service-territory map with DAC / non-DAC areas and layers.
Reporting year	Switch between years (e.g. 2023, 2024) to view that year's figures.
Data ingestion / editor	Add, edit, or remove values; this is where changes are made and saved.

4. Editing data (ingestion)

All editing happens in the dashboard's own editor - there are no separate forms to fill in. Create, edit, and delete all save to Dataverse.

3. **Pick the table and year.** Go to the section/table you want, and make sure the right reporting year is selected.
4. **Make your change.** Edit a cell, add a row, or remove a row directly in the grid. To add or remove a whole reporting year, use the add/remove-year control.
5. **Save.** Click Save. You will be asked for your name and email - this records who made the change.
6. **Confirm.** The change is written to Dataverse and an audit entry is recorded automatically. Reopen or hard-refresh and your change will still be there.

Reset to baseline: Each table can be reset to its original baseline values if you need to undo edits and start over.

What gets recorded with every change: who made it (name + email), when, and what changed. This is the report's audit trail.

5. Did my change save correctly?

Two quick checks confirm a real, durable save (versus a temporary local one):

- After saving, **hard-refresh** (Ctrl+F5) or reopen the app - the change should still be present.
- If you open the browser console (F12), you should see `backend = Dataverse`. If it instead says `backend = localStorage`, you are not connected to the shared database (this is expected only for local previews) - see troubleshooting in Part 2.

Part 2 - Technical reference

6. Where everything lives

- **The code (source of truth):** the GitHub repo, folder `Coned/CLCPA/ExecutiveDashboard_Test/`. Always edit here.
- **The running copy (in Power Apps):** the same files uploaded into Dataverse as web resources under `cr2bf_dactest/`. This is what the app actually serves.

Golden rule: Always edit in the repo, then deploy - never hand-edit the web resource directly in Power Apps. Editing both places separately makes them drift apart and you lose track of which copy is correct.

7. Which file controls what

File	What it controls
<code>ExecutiveDashboard.html</code>	The dashboard page - overall structure and layout.
<code>app.js</code>	The behavior / logic - how data loads from Dataverse, the ingestion editor, charts, and saving. Contains the “Storage” section that talks to Dataverse (edit carefully).
<code>styles.css</code>	The look - colors, fonts, spacing.
<code>payload.json</code>	The dashboard’s structure - table definitions, column headers, charts, KPIs, sections, years. Note: the editable table NUMBERS live in Dataverse, not here.
<code>index.html</code>	The password gate (entry screen).
<code>map_payload.json</code> <code>Data/service_territories.geojson</code>	The map data. The map is drawn by the dashboard itself (Leaflet); both files must be present as web resources for the map and its layers to render.
<code>logo/ ...</code>	Logo images.

8. How storage works (the facade)

The UI never talks to Dataverse directly. A **Storage facade** inside `app.js` sits between them, so the backend can change without touching the UI. At startup it auto-detects where it is running:

- Served from the org domain (inside Power Apps): it uses `window.location.origin` as the Dataverse base URL and authenticates with same-origin session cookies - no helper script needed. Console: `backend = Dataverse`.
- No Dataverse context (e.g. localhost): it falls back to `localStorage`. Console: `backend = localStorage (no Dataverse context)`.

The app reads structure from `payload.json` and reads/writes the actual numbers through the facade. See the **Data Schema** document for the Dataverse tables and the **System Flow** document for the full runtime flows.

9. Editing and deploying - step by step

7. **Open the project in VS Code** - the `ExecutiveDashboard_Test` folder of the repo.
8. **Edit the file(s)** you need and save.
9. **(Optional) Preview locally** - run a local web server and open it. Locally the app uses the localStorage fallback (`backend = localStorage`), which is fine for checking look and layout, but you will not see live Dataverse data.
10. **Commit & push to GitHub** - keeps version history and a backup.
11. **Deploy to the app** - get the changed file(s) into Dataverse as web resources under `cr2bf_dactest/` and Publish. Manually: Power Apps → Solutions → CLCPA DAC Dashboard → Web resources → open the file → upload → Save → Publish.
12. **See your changes** - open the app and hard-refresh (Ctrl+F5) or use an incognito window. Power Apps caches web resources aggressively, so a normal refresh often shows the old version.

Structure changes: Editing `payload.json` changes definitions (tables, headers, charts) - not the live numbers. If a structural change also affects the baseline data, re-sync Dataverse afterward (re-run the data loader) so the tables match.

10. Troubleshooting

Symptom	What to do
Console shows <code>backend = localStorage</code>	You are not connected to Dataverse. Open the app from inside the model-driven Power Apps app (or via the org-domain URL), not a local copy. Local previews always use localStorage by design.
Changes don't appear after deploy	Power Apps caches web resources. Hard-refresh (Ctrl+F5) or use an incognito window. Confirm the file was Published, not just Saved.
Map or layers don't draw	Confirm <code>map_payload.json</code> and <code>Data/service_territories.geojson</code> are uploaded as web resources alongside the dashboard.
A table is blank	There may be no record for that table-year. A clear empty state (not stale data) is expected; ingest the data via the editor.
Edits seem lost on reload	Verify the save completed (you entered name/email) and that the console reports the Dataverse backend; a localStorage save will not persist across browsers/sessions.

Symptom	What to do
Repo and running copy disagree	The web resource was hand-edited. Re-deploy from the repo (source of truth) to bring them back in sync.

11. Reference details

Item	Value
Environment	Clara Fortuna Dev (Developer environment with Dataverse)
Org URL	https://org9076e69b.crm.dynamics.com
Publisher prefix	cr2bf_
Hosted gate URL	.../WebResources/cr2bf_dactest/index.html password: dacdemo
Model-driven app	ExecutiveDashboard_test
Solution	CLCPA DAC Dashboard
Source of truth	Coned/CLCPA/ExecutiveDashboard_Test/ (GitHub)

Dataverse tables

Display name	Logical name	Role
DAC Ingest Test - Table Data	cr2bf_dacingesttesttabledata1	One record per table-year; grid stored as JSON.
DAC Ingest Test Change History	cr2bf_dacingesttestchangehistory	Audit log: WHO / WHEN / what changed.
DAC Ingest Test Reporting Year	cr2bf_dacingesttestreportingyear	Available reporting years.

12. Glossary

- **Dataverse** - Microsoft's shared online database behind the Power Platform.
- **Power Apps / Power Platform** - Microsoft's environment for building and hosting business apps.
- **Model-driven app** - a standard Power Platform app shell; hosting the dashboard inside it guarantees the Dataverse connection is available.
- **Web resource** - a file (HTML, JS, CSS, data) uploaded into Dataverse so the app can be served from there.
- **localStorage** - a small storage area inside a single browser; not shared, not durable. Used only as a local fallback.
- **Record** - one row in a Dataverse table.

- **Memo (multiple lines of text)** - a Dataverse column type that holds large text; used here to store each table's grid as JSON.
- **DAC** - Disadvantaged Community, the NYSEDA census-tract designation. Distinct from "low-income" (an EAP participant).

Related documents: **System Flow** (architecture and runtime/deploy flows) and **Data Schema** (deployed model + canonical star schema).